

**OCEANSENSE: AN IOT-BASED HYBRID SYSTEM FOR REAL-TIME DETECTION
AND TRACKING OF MARINE DEBRIS**

25-26J-002

DESIGN DOCUMENT (DD)

Jayasak D L C – IT22908360

Supervisor – Ms. Dinithi Pandithage

BSc (Hons) in Information Technology Specializing in Computer Systems & Network
Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

11 January 2026

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 References.....	4
2. Overall Description.....	4
2.1 Product Perspective.....	4
2.2 System Interfaces	4
2.2.1 Hardware Interfaces	4
2.2.2 Software Interfaces	4
2.2.3 Communication Interfaces	5
2.3 User Interfaces	5
2.4 Operating Environment.....	6
2.5 Product Functions	6
2.6 Constraints	6
2.7 Assumptions and Dependencies.....	6
3. Design Specifications.....	7
3.1 High-Level Architectural Design.....	7
3.2 Architectural Design	7
3.3 Component Selection and Justification.....	8
3.4 Self-Healing Mechanism Design	9
3.6 Performance Requirements	9
3.7 Design Constraints	9
4. Supporting Information.....	10
4.1 Appendices.....	10
Receiver Function.....	10
Transmitter Functions	11

1.Introduction

1.1 Purpose

This Design Document (DD) presents the detailed design of the OCEANSENSE system, a decentralized IoT-based swarm platform intended for marine debris monitoring. The primary purpose of this document is to demonstrate a clear understanding of the system architecture, hardware design, communication mechanisms, and self-healing behavior.

1.2 Scope

The scope of this Design Document includes:

- Hardware and embedded-system architecture of a swarm node
- LoRa-based wireless swarm communication design
- GSM-assisted fallback mechanism for last-known location retrieval
- GPS and compass-assisted autonomous navigation
- Self-healing mechanism under communication failure
- Design constraints, risks, and mitigation strategies

1.3 Definitions, Acronyms, and Abbreviations

Terms	Description
LoRa	Long Range, low-power wireless communication technology
GSM	Global System for Mobile Communication
GPS	Global Positioning System
SPI	Serial Peripheral Interface
FSM	Finite State Machine
I2C	Inter-Integrated Circuit

1.4 References

- Arduino / ESP32 Technical Datasheets
- SX1278 LoRa Module Datasheet
- HMC5883L Digital Compass Datasheet
- RP Module Guidelines – CSNE
- IEEE Literature on Marine IoT and Swarm Systems

2. Overall Description

2.1 Product Perspective

OCEANSENSE is designed as a decentralized swarm system with no central controller. Each node operates autonomously while coordinating with neighboring nodes through LoRa communication. The system is resilient to individual node or communication failures through its self-healing design.

2.2 System Interfaces

2.2.1 Hardware Interfaces

- Microcontroller: Arduino / ESP32
- LoRa transceiver connected via SPI
- Digital compass connected via I2C
- GPS and GSM modules connected via UART
- DC motors connected via motor driver (PWM control)
- LEDs and push buttons via GPIO

2.2.2 Software Interfaces

- Arduino IDE for firmware development
- Embedded C/C++ firmware
- LoRa and Wire (I2C) libraries

2.2.3 Communication Interfaces

- Primary: LoRa peer-to-peer broadcast communication (915 MHz)
- Secondary (Fallback): GSM used for retrieving last-known location only

2.3 User Interfaces

- Serial Monitor output for diagnostics and demonstration

```
15:20:22.694 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
15:20:23.415 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
15:20:24.103 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
15:20:24.792 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
15:20:25.479 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
15:20:26.201 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
15:20:26.889 -> ⚡ Sent Packet: NODE=3,LAT=12.971600,LON=77.594597,STAT=ACTIVE
```

Figure 1 - Serial monitor Output from LoRa transmitter

```
15:16:11.912 -> Navigating to swarm. Heading: 0.00
15:16:11.945 -> Navigating to swarm. Heading: 0.00
15:16:11.978 -> Navigating to swarm. Heading: 0.00
15:16:12.014 -> ⚡ Bot initialized. Waiting for LoRa packets...
```

Figure 1- Monitoring the receiving signal from LoRa Transmitter

```
15:16:55.802 -> Navigating to swarm. Heading: 0.00
15:17:02.608 -> ⚡ Bot initialized. Waiting for LoRa packets...
15:17:07.613 -> Lost LoRa, retrying...
15:17:07.613 -> LoRa failed, entering SELF-HEAL mode...
15:17:07.645 -> Fetching last known location from GSM...
15:17:07.678 -> Last Location: 37.77 -122.42
```

Figure 3- Fetching Last known Location when LoRa fails

```
Output - Serial Monitor X
Message (Enter to send message to 'Arduino Uno' on 'COM4')
15:17:16.328 -> Navigating to swarm. Heading: 0.00
15:17:16.36 -> Compass Heading: 0.00
15:17:16.36 -> Navigating to swarm. Heading: 0.00
15:17:16.42 -> Navigating to swarm. Heading: 0.00
15:17:16.45 -> Navigating to swarm. Heading: 0.00
15:17:16.492 -> Navigating to swarm. Heading: 0.00
15:17:16.524 -> Navigating to swarm. Heading: 0.00
15:17:16.557 -> Navigating to swarm. Heading: 0.00
15:17:16.590 -> Navigating to swarm. Heading: 0.00
15:17:16.622 -> Navigating to swarm. Heading: 0.00
15:17:16.688 -> Navigating to swarm. Heading: 0.00
15:17:16.721 -> Navigating to swarm. Heading: 0.00
15:17:16.754 -> Navigating to swarm. Heading: 0.00
```

Figure 4- Shifting Last know Location from using GSM

- LED indicators representing swarm states (IN_SWARM, RETRY_SWARM, SELF_HEAL_SWARM)

2.4 Operating Environment

The system is intended to operate on marine surfaces and must tolerate node drift, environmental interference, and intermittent communication loss.

2.5 Product Functions

- Periodic transmission and reception of swarm status packets
- Continuous monitoring of communication health
- Detection of LoRa communication loss
- Autonomous transition to self-healing mode
- Navigation toward swarm using GPS and compass data
- Rejoining the swarm after recovery

2.6 Constraints

- Limited power availability
- Environmental signal attenuation
- Limited processing and memory resources
- Cost constraints for scalable deployment

2.7 Assumptions and Dependencies

- All nodes operate on a common LoRa frequency
- Last-known location data is available before communication loss

3. Design Specifications

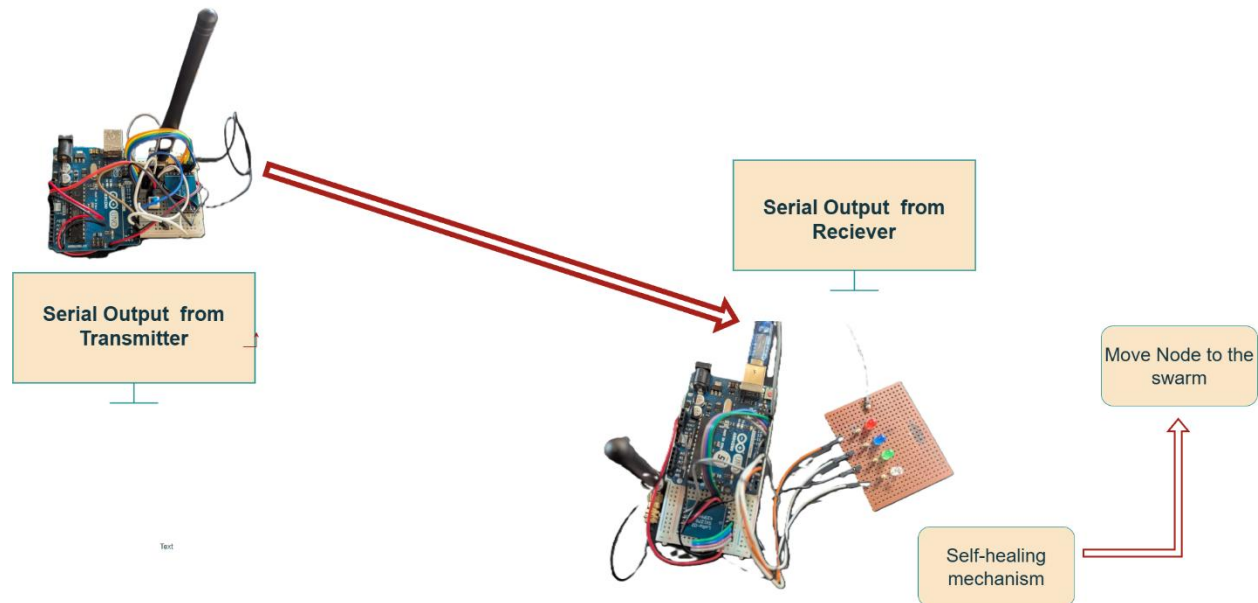
3.1 High-Level Architectural Design

The system follows a **Finite State Machine (FSM)** architecture consisting of three operational states:

1. **IN_SWARM** – Normal operation with active LoRa communication
2. **RETRY_SWARM** – Temporary reconnection attempts after packet timeout
3. **SELF_HEAL_SWARM** – Autonomous navigation-based recovery when communication fails

State transitions are triggered by LoRa packet timeouts and recovery success conditions.

3.2 Architectural Design



3.3 Component Selection and Justification

LoRa Communication Module

LoRa is used as the primary communication technology between swarm nodes due to its:

Long-range communication capability

Low power consumption

Suitability for decentralized, infrastructure-less environments

LoRa enables swarm nodes to broadcast periodic status packets containing node ID, location, and operational state. This allows nodes to maintain swarm cohesion without relying on centralized infrastructure.

LoRa is chosen over Wi-Fi or Bluetooth because the system is designed for outdoor, long-range, and fault-tolerant operation, where conventional short-range communication methods are unreliable.

GSM Module (Fallback Communication)

The GSM module is implemented as a fallback mechanism, not as the primary communication channel.

Its role is to:

Provide access to the last known swarm location when LoRa communication is unavailable

Act as an emergency data source during extended communication failures

GSM is intentionally used only in failure scenarios due to:

Higher power consumption

Dependence on cellular infrastructure

This design choice ensures that GSM does not replace LoRa but instead enhances system resilience by supporting self-healing behavior.

Compass Module (HMC5883L)

The digital compass provides heading information required for directional navigation during self-healing mode.

It enables the swarm node to:

Maintain orientation awareness

Navigate toward the last known swarm location when isolated

The compass communicates via I2C, reducing wiring complexity and allowing efficient sensor integration.

3.4 Self-Healing Mechanism Design

When LoRa communication is lost beyond a predefined timeout:

- The node enters `RETRY_SWARM` mode
- If reconnection fails, the node switches to `SELF_HEAL_SWARM`
- The last-known location is retrieved via GSM or stored memory
- Compass heading and GPS data are used to navigate toward the swarm

This mechanism ensures **graceful degradation** rather than system failure.

3.6 Performance Requirements

- Communication loss detection within predefined timeout
- Navigation response within seconds after state transition
- Minimal power usage during idle and recovery states

3.7 Design Constraints

- Limited bandwidth of LoRa communication
- Hardware pin and memory limitations

4. Supporting Information

4.1 Appendices

- State transition description
- Sample LoRa packet format: NODE=ID,LAT=xx.xxxxxx,LON=yy.yyyyyy,STAT=ACTIVE
- Pin configuration tables

Receiver Function

```
sketch_janlino
1 #include <Wire.h>
2 #include <SPI.h>
3 #include <LoRa.h>
4
5 // ===== PINS =====
6 #define LORA_SS 10
7 #define LORA_RST 9
8 #define LORA_DIO0 2
9
10 #define LED_BLUE 4
11 #define LED_RED 5
12 #define LED_GREEN 6
13 #define LED_WHITE 7
14
15 #define MOTOR_LEFT 8
16 #define MOTOR_RIGHT 9
17
18 #define HMC5883L_ADDR 0x1E
19
20 // ===== TIMERS =====
21 unsigned long lastLoraTime = 0;
22 unsigned long retryStart = 0;
23 unsigned long lastCompassPrint = 0;
24
25 // ===== SWARM STATE =====
26 enum SwarmState { IN_SWARM, RETRY_SWARM, SELF_HEAL_SWARM };
27 SwarmState currentState = RETRY_SWARM;
28
29 // ===== BLINK STRUCT =====
30 struct Blink { unsigned long interval; unsigned long lastToggle; bool state; };
31 Blink blueBlink = {200,0,LOW};

15:17:16.754 -> Navigating to swarm. Heading: 0.00

56 if(!LoRa.begin(015E6)){
57   Serial.println("LoRa init failed!");
58   while(true);
59 }
60
61 // Compass init
62 Wire.begin();
63 Wire.beginTransmission(HMC5883L_ADDR);
64 Wire.write(0x00); Wire.write(0x70); Wire.endTransmission();
65 Wire.beginTransmission(HMC5883L_ADDR);
66 Wire.write(0x01); Wire.write(0x20); Wire.endTransmission();
67 Wire.beginTransmission(HMC5883L_ADDR);
68 Wire.write(0x02); Wire.write(0x00); Wire.endTransmission();
69
70 Serial.println("Bot initialized. Waiting for LoRa packets...");
71 lastLoraTime = millis();
72 retryStart = millis();
73 }
74
75 // ===== GSM =====
76 void setupGSM() { digitalWrite(LED_RED,LOW); digitalWrite(LED_WHITE,HIGH); }
77
78 // ===== GSM =====
79 void getLastLocationFromGSM(){
80   Serial.println("Fetching last known location from GSM...");
81   lastLatitude = 37.7749; // placeholder
82   lastLongitude = -122.4194; // placeholder
83   Serial.print("Last location: "); Serial.print(lastLatitude); Serial.print(", "); Serial.println(lastLongitude);
84 }
85
86 // ===== COMPASS =====
87 float getCompassHeading(){
88   Wire.beginTransmission(HMC5883L_ADDR);
89   Wire.write(0x02); Wire.endTransmission();
90   Wire.requestFrom(HMC5883L_ADDR,6);
91   if(Wire.available() >= 6){
92     int16_t x = Wire.read() << 8 | Wire.read();
93     Wire.read(); Wire.read(); // 2 skipped
94     int16_t y = Wire.read() << 8 | Wire.read();
95     float heading = atan2((float)y, (float)x) * 180.0 / 3.14159265;
96     if(heading < 0) heading += 360.0;
97     return heading;
98   }
99   return 0.0;
100 }
101
102 }
103
```

```

86     }
87
88     // ===== LoRa RECEIVE =====
89     int packetSize = LoRa.parsePacket();
90     if(packetSize){
91         String msg = "";
92         while(LoRa.available()){
93             msg += (char)LoRa.read();
94         }
95         msg.trim();
96         Serial.print("LoRa Received: "); Serial.println(msg);
97
98         if(msg == "I am in the swarm"){
99             currentState = IN_SWARM;
100             lastIoraTime = now;
101             retryStart = now;
102             digitalWrite(LED_RED, LOW);
103             digitalWrite(LED_GREEN, LOW);
104             digitalWrite(LED_WHITE, LOW);
105             Serial.println("IN SWARM");
106         }
107     }
108 }

```

Transmitter Functions

```

digitalWrite(LED_GREEN, LOW);
digitalWrite(LED_BLUE, HIGH);

LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
if (!LoRa.begin(915E6)) {
    Serial.println("❌ LoRa init failed");
    while (1);
}

Serial.println("✅ LoRa Swarm Transmitter Ready");
}

```

```

#include <SPI.h>
#include <LoRa.h>

/* ===== LoRa PINS ===== */
#define LORA_SS 10
#define LORA_RST 9
#define LORA_DIO0 2

/* ===== IO ===== */
#define BUTTON_PIN 3
#define LED_GREEN 4
#define LED_BLUE 5

/* ===== NODE CONFIG ===== */
#define NODE_ID 3

```

